

Name : Narendra Kumar S

Date : 20-08-2025

Task :

1. To the existing books table, add additional column called price and add at least ten entries with some price values.

1.1 Create a post request 'GetBooksOrderedByPrice()' with a boolean parameter

1.1.1 when parameter is true, then sort by price ascending order

1.1.2 when parameter is false, then sort by price descending order

1.2 Create a get request 'GetTop5Books()' which will return top 5 costliest books

1.3 Create a get request 'GetCostDetails()' which will return the total price and average price of all books

Solution:

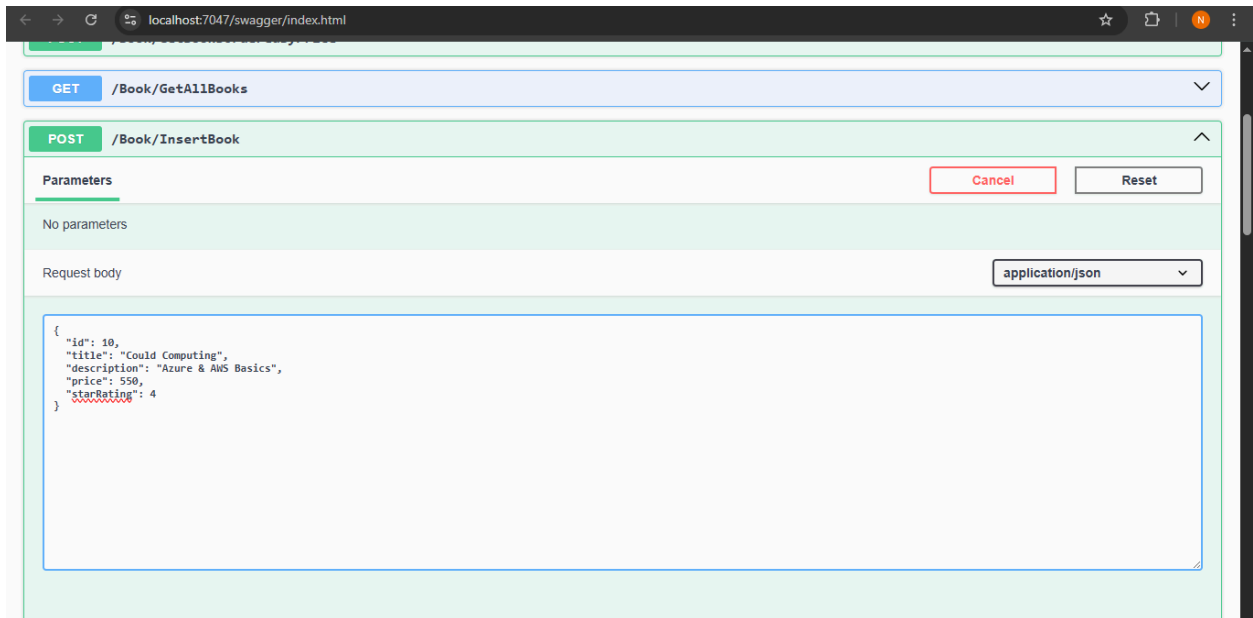
1.Adding the Price Column in the Model Class and Table

using System.ComponentModel.DataAnnotations;

```
namespace Day2Api.Model
{
    public class Book
    {
        public int Id { get; set; }

        [StringLength(100,MinimumLength =5,ErrorMessage ="Length should be minimum
5")]
        public string Title { get; set; }
        public string Description { get; set; }
        public int Price { get; set; }
        [Range(0,5)]
        public int StarRating { get; set; }
    }
}
```

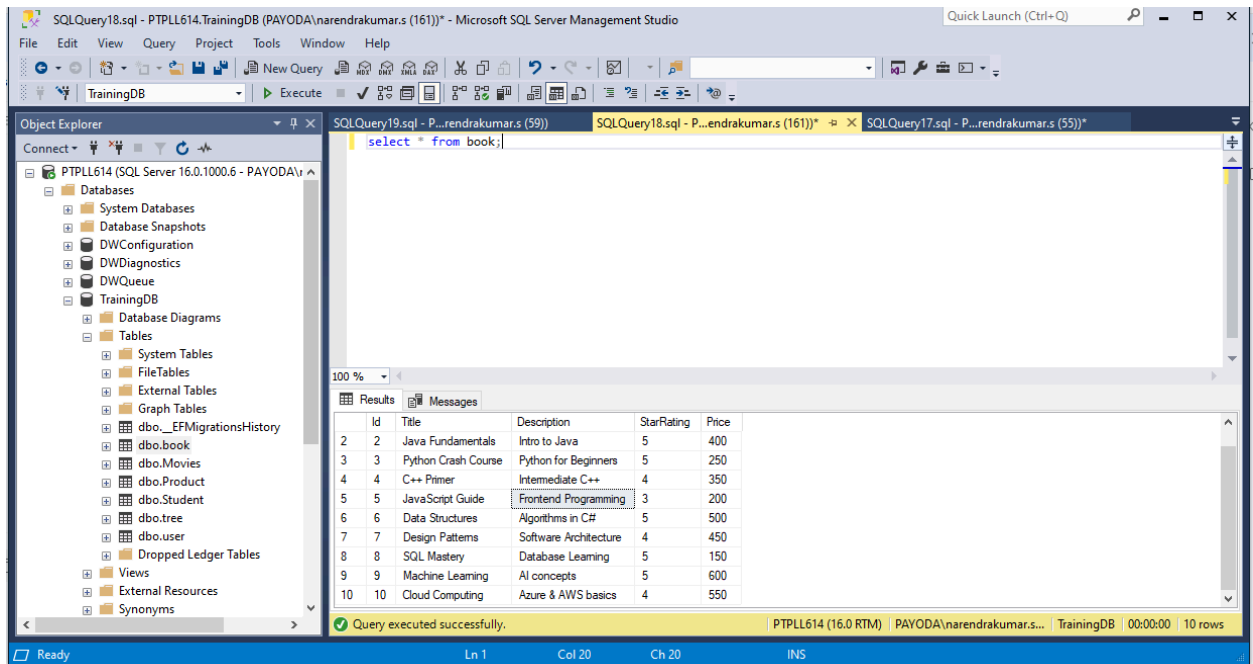
2.inserting the records in the table



The image shows the Swagger UI for a REST API. The selected endpoint is **POST /Book/InsertBook**. The parameters section indicates "No parameters". The request body is set to **application/json** and contains the following JSON:

```
{  "id": 10,  "title": "Cloud Computing",  "description": "Azure & AWS Basics",  "price": 550,  "starRating": 4}
```

Buttons for "Cancel" and "Reset" are visible in the top right of the parameters section.



The screenshot shows Microsoft SQL Server Management Studio with the following details:

- Object Explorer:** Shows the database structure for **PTPL614** (SQL Server 16.0.1000.6 - PAYODA). The **TrainingDB** database is expanded, showing tables like **dbo.book**, **dbo.Movies**, **dbo.Product**, **dbo.Student**, **dbo.tree**, **dbo.user**, and **External Resources**.
- SQL Query Editor:** Contains the query `select * from book;`.
- Results:** Displays the following data:

Id	Title	Description	StarRating	Price
2	Java Fundamentals	Intro to Java	5	400
3	Python Crash Course	Python for Beginners	5	250
4	C++ Primer	Intermediate C++	4	350
5	JavaScript Guide	Frontend Programming	3	200
6	Data Structures	Algorithms in C#	5	500
7	Design Patterns	Software Architecture	4	450
8	SQL Mastery	Database Learning	5	150
9	Machine Learning	AI concepts	5	600
10	Cloud Computing	Azure & AWS basics	4	550

The status bar at the bottom indicates: **Query executed successfully.** | **PTPL614 (16.0 RTM)** | **PAYODA\ Narendra Kumar.s...** | **TrainingDB** | **00:00:00** | **10 rows**

Post Request Methods:

```
using Day2Api.Model;
using Day6_Web.context;
using Microsoft.AspNetCore.Mvc;
using System;
```

```
namespace Day2Api.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("[controller]")]
```

```
    public class BookController : Controller
```

```
    {
```

```
        AppDBcontext appDBcontext;
```

```
        public BookController(AppDBcontext context)
```

```
        {
```

```
            appDBcontext = context;
```

```
        }
```

```
        [HttpPost("GetBooksOrderedByPrice")]
```

```
        public ActionResult<IEnumerable<Book>> GetBooksOrderedByPrice(bool ascending)
```

```
        {
```

```
            var books = appDBcontext.book.ToList();
```

```
            if (ascending)
```

```
            {
```

```
                books = books.OrderBy(b => b.Price).ToList();
```

```
            }
```

```
            else
```

```
            {
```

```
                books = books.OrderByDescending(b => b.Price).ToList();
```

```
            }
```

```
            return Ok(books);
```

```
        }
```

```
        [HttpGet("GetAllBooks")]
```

```
        public IEnumerable<Book> GetAllBooks()
```

```
        {
```

```
            List<Book> list = new List<Book>();
```

```

list.Add(new Book
{
    Title = "C#",
    Description = "Programing Language",
    StarRating = 4
});
list.Add(new Book
{
    Title = "Java",
    Description = "High Level Programing Language",
    StarRating = 5
});
return list;
}

```

```

[HttpPost("InsertBook")]
public IActionResult AddBooks(Book book)
{
    appDBcontext.book.Add(book);
    appDBcontext.SaveChanges();
    return Ok("Added the book to store");
}

```

```

[HttpGet("GetBooksFromDB")]
public IEnumerable<Book> GetBooks()
{
    var list = appDBcontext.book.ToList();
    return list;
}

```

```

[HttpDelete("DeleteBookFromDB")]
public IActionResult DeleteBook(int id)
{
    var book = appDBcontext.book.FirstOrDefault(b => b.Id == id);

    if (book == null)
    {
        return NotFound($"Book with Id = {id} not found");
    }

    appDBcontext.book.Remove(book);
    appDBcontext.SaveChanges();
}

```

```

        return Ok($"Book with Id = {id} deleted successfully");
    }

    [HttpGet("GetTop5Books")]
    public ActionResult<IEnumerable<Book>> GetTop5Books()
    {
        var books = appDBcontext.book
            .OrderByDescending(b => b.Price)
            .Take(5)
            .ToList();

        return Ok(books);
    }

    [HttpGet("GetCostDetails")]
    public ActionResult<object> GetCostDetails()
    {
        var books = appDBcontext.book.ToList();

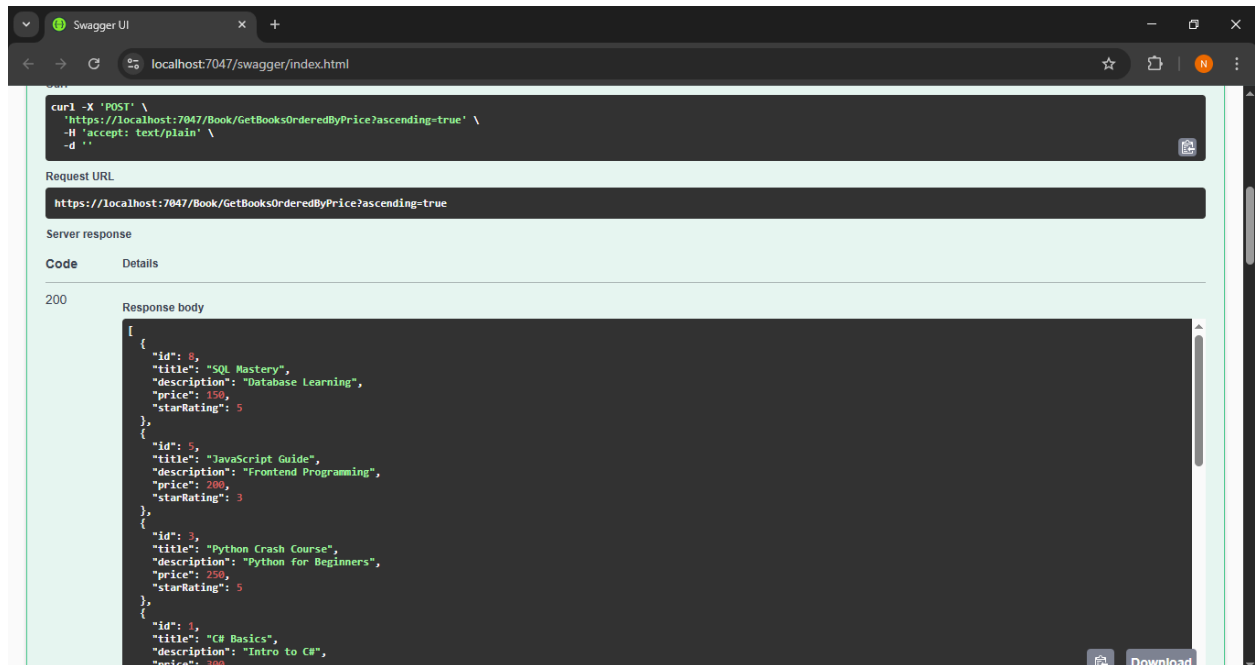
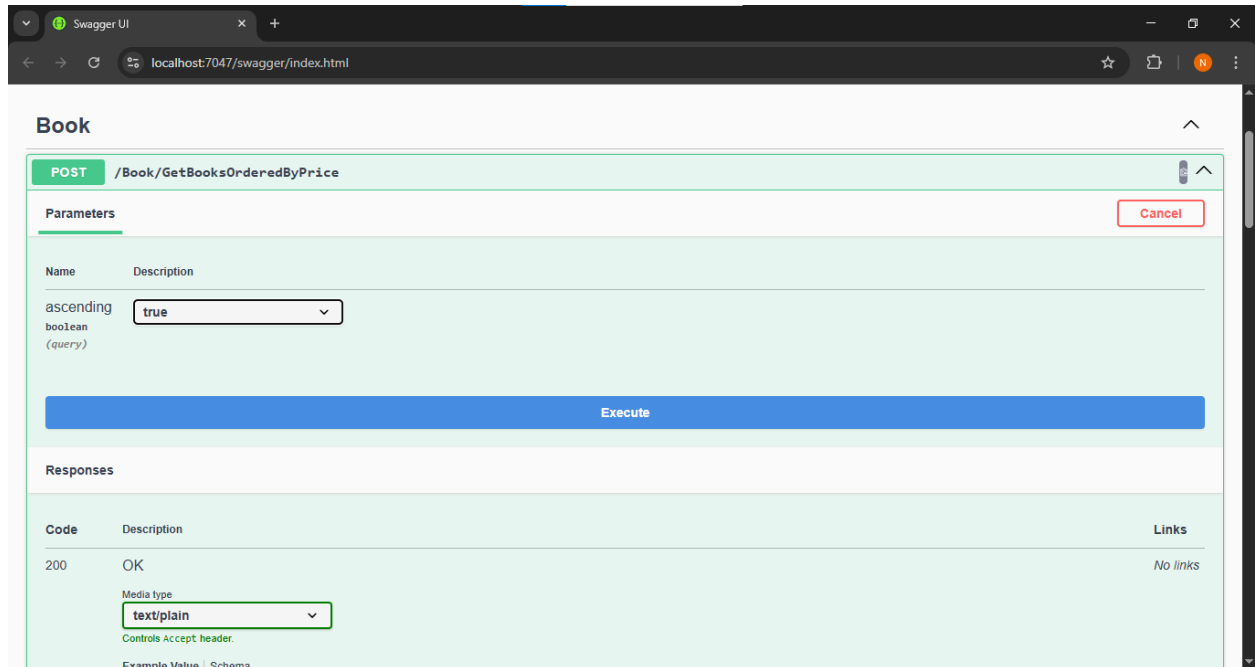
        if (!books.Any())
        {
            return NotFound("No books available to calculate cost details.");
        }

        var totalPrice = books.Sum(b => b.Price);
        var averagePrice = books.Average(b => b.Price);

        return Ok(new
        {
            TotalPrice = totalPrice,
            AveragePrice = averagePrice
        });
    }
}

```

Output for value is **True** :



Result From Query:

```
[
  {
    "id": 8,
    "title": "SQL Mastery",
    "description": "Database Learning",
    "price": 150,
```

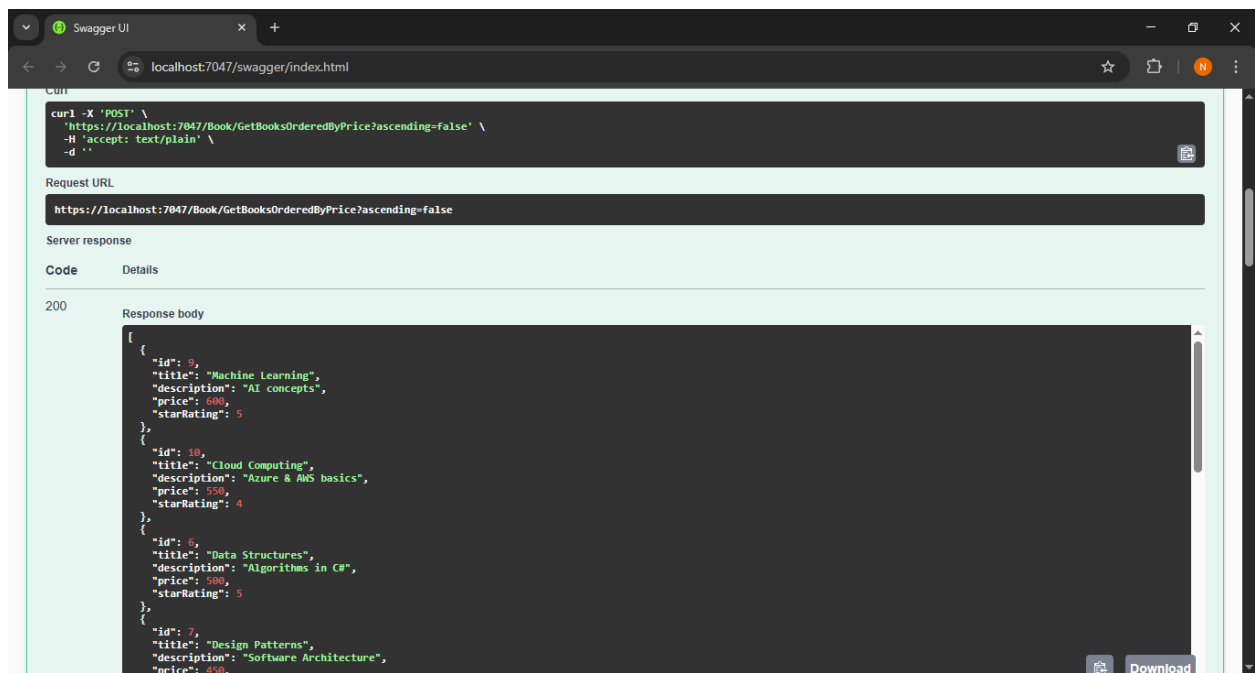
```
    "starRating": 5
  },
  {
    "id": 5,
    "title": "JavaScript Guide",
    "description": "Frontend Programming",
    "price": 200,
    "starRating": 3
  },
  {
    "id": 3,
    "title": "Python Crash Course",
    "description": "Python for Beginners",
    "price": 250,
    "starRating": 5
  },
  {
    "id": 1,
    "title": "C# Basics",
    "description": "Intro to C#",
    "price": 300,
    "starRating": 4
  },
  {
    "id": 4,
    "title": "C++ Primer",
    "description": "Intermediate C++",
    "price": 350,
    "starRating": 4
  },
  {
    "id": 2,
    "title": "Java Fundamentals",
    "description": "Intro to Java",
    "price": 400,
    "starRating": 5
  },
  {
    "id": 7,
    "title": "Design Patterns",
    "description": "Software Architecture",
    "price": 450,
    "starRating": 4
  },
  {
    "id": 6,
    "title": "Data Structures",
    "description": "Algorithms in C#",
    "price": 500,
    "starRating": 5
  },
  {
    "id": 10,
    "title": "Cloud Computing",
    "description": "Azure & AWS basics",
```

```

    "price": 550,
    "starRating": 4
  },
  {
    "id": 9,
    "title": "Machine Learning",
    "description": "AI concepts",
    "price": 600,
    "starRating": 5
  }
]

```

Output for value is **False**:



Result From Query Execution:

```

[
  {
    "id": 9,
    "title": "Machine Learning",
    "description": "AI concepts",
    "price": 600,

```

```
    "starRating": 5
  },
  {
    "id": 10,
    "title": "Cloud Computing",
    "description": "Azure & AWS basics",
    "price": 550,
    "starRating": 4
  },
  {
    "id": 6,
    "title": "Data Structures",
    "description": "Algorithms in C#",
    "price": 500,
    "starRating": 5
  },
  {
    "id": 7,
    "title": "Design Patterns",
    "description": "Software Architecture",
    "price": 450,
    "starRating": 4
  },
  {
    "id": 2,
    "title": "Java Fundamentals",
    "description": "Intro to Java",
    "price": 400,
    "starRating": 5
  },
  {
    "id": 4,
    "title": "C++ Primer",
    "description": "Intermediate C++",
    "price": 350,
    "starRating": 4
  },
  {
    "id": 1,
    "title": "C# Basics",
    "description": "Intro to C#",
    "price": 300,
    "starRating": 4
  },
  {
    "id": 3,
    "title": "Python Crash Course",
    "description": "Python for Beginners",
    "price": 250,
    "starRating": 5
  },
  {
    "id": 5,
    "title": "JavaScript Guide",
    "description": "Frontend Programming",
```

```

        "price": 200,
        "starRating": 3
    },
    {
        "id": 8,
        "title": "SQL Mastery",
        "description": "Database Learning",
        "price": 150,
        "starRating": 5
    }
]

```

2. Returning the top 5 Costliest book from the table

Solution:

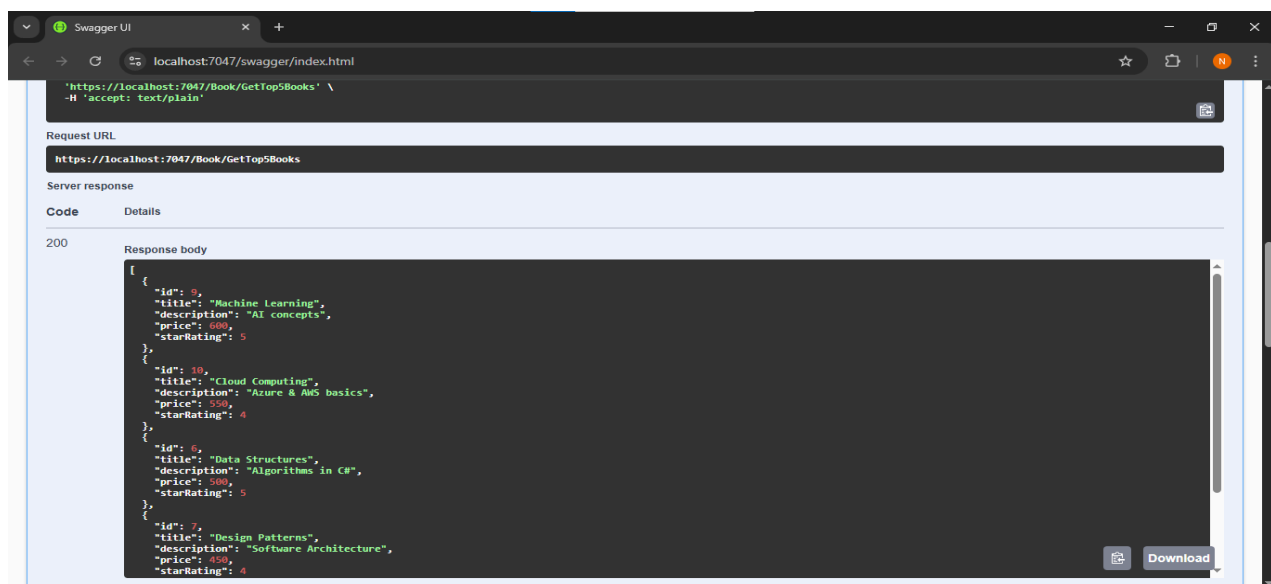
```

[HttpGet("GetTop5Books")]
public ActionResult<IEnumerable<Book>> GetTop5Books()
{
    var books = appDbContext.book
        .OrderByDescending(b => b.Price)
        .Take(5)
        .ToList();

    return Ok(books);
}

```

Output:



Output form the query:

```
[
  {
    "id": 9,
    "title": "Machine Learning",
    "description": "AI concepts",
    "price": 600,
    "starRating": 5
  },
  {
    "id": 10,
    "title": "Cloud Computing",
    "description": "Azure & AWS basics",
    "price": 550,
    "starRating": 4
  },
  {
    "id": 6,
    "title": "Data Structures",
    "description": "Algorithms in C#",
    "price": 500,
    "starRating": 5
  },
  {
    "id": 7,
    "title": "Design Patterns",
    "description": "Software Architecture",
    "price": 450,
    "starRating": 4
  },
  {
    "id": 2,
    "title": "Java Fundamentals",
    "description": "Intro to Java",
    "price": 400,
    "starRating": 5
  }
]
```

3.Get total cost and Average cost of the book

Solution:

```
[HttpGet("GetCostDetails")]  
public ActionResult<object> GetCostDetails()
```

```

{
    var books = appDBcontext.book.ToList();

    if (!books.Any())
    {
        return NotFound("No books available to calculate cost details.");
    }

    var totalPrice = books.Sum(b => b.Price);
    var averagePrice = books.Average(b => b.Price);

    return Ok(new
    {
        TotalPrice = totalPrice,
        AveragePrice = averagePrice
    });
}

```

Output :

The screenshot shows the Swagger UI interface in a web browser. The browser's address bar displays 'localhost:7047/swagger/index.html'. The UI has a top bar with 'Execute' and 'Clear' buttons. Below this, the 'Responses' section is expanded, showing the details of a 200 status code response. The 'Curl' section contains the command: `curl -X 'GET' \ 'https://localhost:7047/Book/GetCostDetails' \ -H 'accept: */*'`. The 'Request URL' section shows 'https://localhost:7047/Book/GetCostDetails'. The 'Server response' section is divided into 'Code' (200) and 'Details'. The 'Response body' is displayed as a JSON object: `{ "totalPrice": 3750, "averagePrice": 375 }`. The 'Response headers' section shows: `content-type: application/json; charset=utf-8`, `date: Wed, 20 Aug 2025 12:51:08 GMT`, and `server: Kestrel`. At the bottom, there is a table with columns 'Code', 'Description', and 'Links'.